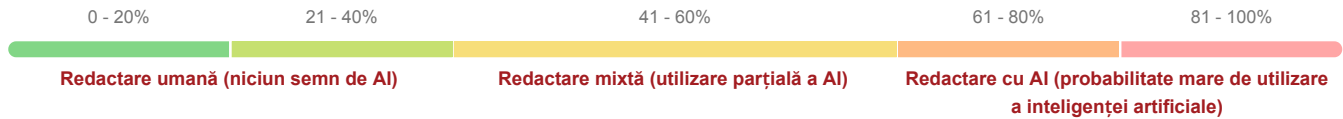


Rezultatele arată probabilitatea ca textul să fi fost creat de un instrument AI precum ChatGPT, Bing, Claude, Google Gemini, LLaMA, DeepSeek, Grok, Jasper sau altele.

Textul indică probabilitatea de utilizare a AI pentru fragmente separate ale documentului și le grupează în 5 intervale, în conformitate cu legenda.



Metadate

Titlu
main_lite_check

Autor
Krisztian T

Lungimea în cuvinte
10636

Lungimea în caractere
71739

Data Raportului
2026-06-30

Data editării

ID Document
334449043

Indicator Conținut AI

0%

Detalii

Mai jos este lista cu fragmentele sortate în ordine descrescătoare de la fragmentele cu cel mai mare procent de probabilitate AI până la fragmentele cu cel mai mic procent.

NR.	FRAGMENT	DETECTAREA CONȚINUTULUI AI	NUMĂRUL DE CUVINTE
-----	----------	----------------------------	--------------------

I. rész

PremadeGraph rendszer és gráfanalitika

1

1. fejezet

Bevezetés

1.1 Problémafelvétel

A kompetitív online játékok egyik visszatérő problémája, hogy a résztvevők nagyon erős véleményeket fogalmaznak meg egymás teljesítményéről, miközben ehhez rendszerint csak részleges, szubjektív és rövid távú benyomások állnak rendelkezésre. Egy elvesztett mérkőzés után gyorsan megjelenik a búnbakkeresés, de sokkal ritkábban történik meg az, hogy a játékosok valóban adatok alapján próbálják megérteni, mi történt a meccsben, hogyan teljesítettek ők maguk, és hogyan helyezhető el ez a teljesítmény egy tágabb közösségi hálózatban.

A League of Legends különösen érdekes terep ehhez, mert egyszerre tartalmaz rendkívül gazdag mérkőzésadatokat, ismétlődő csapattársi és ellenfélkapcsolatokat, valamint olyan társas mintázatokat, amelyek gráfként ábrázolhatók és elemezhetők. Ha egy játékos hosszabb időn keresztül ugyanazokkal az emberekkel kerül egy meccsbe (akár szövetségesként, akár ellenfélként), akkor ebből egy olyan kapcsolati háló építhető fel, amely túlmutat az egyetlen mérkőzés pillanatnyi statisztikáin.

Ebből a felismerésből indult a dolgozat. Cél: valós mérkőzéstörténetből játékoshálót építeni, ezt a hálót gráfelméleti szemlélettel elemezni, majd az eredményeket vizuálisan értelmezhető front-end-felületen megjeleníteni.

1.2 Motiváció

A projekt mögött részben személyes indíttatás húzódik. Rangsorolt meccsek után visszatérő kérdés volt, hogy a sorozatos negatív találkozások véletlenek-e vagy van mögöttük valami szerkezet: esetleg ugyanazok az emberek kerülnek újra és újra egymás mellé, csak ezt a matchmaking-felület nem mutatja meg. Ez önmagában persze nem tudományos kérdés, de kiindulópontként elég volt. Ha a Riot API részletes meccsadatokat ad, és ezek ismétlődő játé-

koskapcsolatokat tartalmaznak, akkor elvileg gráf építhető belőlük, és ha gráf van, már van mit elemezni.

2

A projekt hatóköre a fejlesztés során fokozatosan bővült. Ami gyors adatvizualizációként indult, az külön adatgyűjtő réteggé, normalizált pontszámítási pipeline-ná, SQLite-alapú perzisztenciává, Rust futtatókörnyezetté és többoldalas React frontenddé alakult. A Python réteg adatgyűjtési és kompatibilitási szerepben maradt. Minden bővítés egy konkrét technikai korlátot oldott meg, és nem előre megtervezett kiterjesztés volt.

Egy ponton a gráfelemzési pipeline klaszterprofil-exportokat kezdetgyártani, és felmerült, hogy ezek a profilok ágens-alapú szimulációban is felhasználhatók. Ebből nőtt ki a Tribal NeuroSim, amelyben a két adatkészlet klaszterei azonos szimulációs feltételek mellett versenyeznek egymással.

1.3 A dolgozat célja

A dolgozat egy valós mérkőzésadatokra épülő, több komponensű rendszer megtervezésének, megvalósításának és vizsgálatának dokumentációja. A rendszer League of Legends-mérkőzéstörténetből játékoskapcsolati gráfot épít: az ismétlődő együtt- és egymás ellen játszásból jön létre, amelyhez szerep kör-érzékeny teljesítménymutatók tartoznak. A gráf elemzése kiterjed klaszteresítésre, core-periphery szerkezet vizsgálatára, asszortativitásra és Brandes-féle centralitásra, mindezt Rust-alapú, determinisztikus és újrafuttatható megvalósításban.

A játékos háló elemzése három nézőpontból történik. Hálózati szempontból: a flexset adatkészleten a mag-periféria szerkezet vizsgálható. Metrikai szempontból: a szerep kör-érzékeny opscore és feedscore mérőszámok valós meccs adatokból számolódnak. Rendszertechnikai szempontból: Python, SQLite, Node/Express, Rust és React együttesének minden komponense konkrét feladatot tölt be, amelyre a többi réteg nem alkalmas.

A pipeline minden lépése konkrét bemenetekkel és kimenetekkel újrafuttatható: az adatgyűjtéstől a gráfépítésen át a klaszteresítésig és a szimulációig. A Flex Queue és a SoloQ adatkészlet párhuzamos elemzése pontosan ezt teszi mérhetővé: ugyanaz a pipeline két eltérő bemeneten eltérő eredményt ad, és az eltérés a queue-mechanikából értelmezhetővé válik.

1.4 Kutatási kérdések

Négy konkrét kérdés vezeti a vizsgálatot:

K1. Hogyan építhető fel valós League of Legends mérkőzéstörténetből egy olyan játékos háló, amely nem pusztán az együttjátszást, hanem az ellenfélkapcsolatokat is megőrzi?

K2. Milyen módon lehet a játékosok teljesítményét úgy pontozni, hogy a pontozás figyelembe

vegye a tényleges szerep kör, de közben megmaradjon értelmezhetőnek?

K3. Mennyiben indokolt a rendszer különböző rétegeit Python, SQLite, Node.js, Rust és React komponensekre bontani?

K4. Mutat-e a jelenlegi gráf pozitív asszortativitást az opscore és feedscore tekintetében?

3

K5. Miben különbözik a flexset és a soloq kapcsolati szerkezete, és mennyiben írható le a flexset asszociatív core-periphery játékosgráfként?

1.5 Hatókör és korlátok

A projekt viszonylag nagy rendszer, és könnyen csúszhat abba a hibába, hogy minden meglévő ötlet kész eredményként szerepeljen. A dolgozat igyekszik ezt elkerülni. Ami ténylegesen fut és ellenőrizhető, az kész állapotként szerepel. Amit a fejlesztési tervek jövőbeli célként fogalmaznak meg, az nem szerepel kész feature-ként.

1.6 A dolgozat szerkezete

A fejezetek az adatgyűjtéstől a gráfanalitikán át a szimulációs kísérletig haladnak.

Az első fejezetek a kiindulópontot rögzítik: a problémafelvetést, a szakirodalmi háttérrel és a Riot API-re épülő adatforrásokat. Ezt követi a rendszer bemutatása, majd az opscore teljesítménymetrika, a gráfépítés és klaszteresítés mechanizmusa, a Rust futtatókörnyezet, valamint az analitikai modulok. A két adatkészlet (flexset és soloq) összehasonlítása a rendszer legkonkrétabb kutatási eredménye. A dolgozat utolsó egységei az adatbázis-alprojektet és a Tribal NeuroSim szimulációt tárgyalják. Ez a dokumentum az összefoglaló, lite változat; a részletes kifejlesztést tartalmazó bővített verzió [?] az itt csak érintett témákat külön fejezetekben fejti ki.

A 1.1 ábra a jelenlegi frontend egyik belépési pontját mutatja: a meccselemzési és adatgyűjtési felületet.

1.1. ábra: A jelenlegi frontend egyik belépési pontja: meccselemzési és adatgyűjtési felület

4

2. fejezet

Szakirodalmi háttér

2.1 Hálózatkutatás és közösségszerkezet

A gráf mint modell evidens eszköz ismétlődő interakciók vizsgálatához. A videojátékos közösségek esetében különösen izgalmas kérdés, hogy a játékosok közötti ismétlődő interakciók mennyiben alkotnak felismerhető struktúrákat, szubközösségeket, hidakat és lokális mintázatokat.

tokat. Girvan és Newman klasszikus munkája megmutatta, hogy a hálózat élei nem pusztán kapcsolódási elemek, hanem közösségstruktúrát leíró, eltérő fontosságú viszonyok is lehetnek [?]. Fortunato átfogó áttekintése összefoglalja, miért központi probléma a közösségetekeltálás a modern hálózatelemzésben [?].

A jelen projektben a közösségfogalom nem öncélú. Azért fontos, mert a többször ismétlődő kapcsolatokra épülő játékoscsoporthoz értelmezhetőbbé teszik a nagyméretű hálózatot, segítenek elkülöníteni a valós kapcsolati magokat a zajtól és támaszt nyújtanak a későbbi útvonal- és analitikai számításoknak.

2.2 Közösségetekeltálás és modularitás

A korai Python-oldali populációs klaszterezés a NetworkX greedy_modularity_communities eljárását használta, amely a Clauset–Newman–Moore-féle modularitásmaximalizálás megközelítésre épül [?, ?]. Ez tudatos kompromisszum volt: nem a leglátványosabb, hanem gyorsan magyarázható és reprodukálható megoldás. A jelenlegi futásidő gráfállapotot a Rust motor önállóan építi fel a meccs-JSON-fájlokból és az SQLite-adatbázisból.

A modularitás-optimalizáció ismert korlátait sem hagyhatók figyelmen kívül: Fortunato és Barthelemy eredménye szerint a felbontási korlát miatt kisebb, de valós struktúrák a globális optimum érdekében összeolvadhatnak [?]. A python_population klasztereket ezért nem szabad ontológiai “igazi közösségként” kezelni, inkább a választott gráf, súlymodell és heurisztika melletti legjobb, reprodukálható populációs partícióként.

5

2.3 Asszortativitás

Newman munkája az asszortatív keveredést olyan hálózati jelenséggént írja le, amikor a kapcsolódó csomópontok hasonló vagy éppen eltérő tulajdonságúak [?]. A jelen projekt ezt a játékos teljesítménymutatóira alkalmazza: nem az a kérdés, hogy a magas foksámú csúcsok egymáshoz kapcsolódhatnak-e, hanem az, hogy a gráfban összekapcsolt játékosok hasonló opscore vagy feedscore értékeket hordoznak-e.

A dolgozat két gráfmodot különböztet meg: a social-path csak a pozitív, szövetségi támaszú kapcsolatokra épít, a battle-path pedig az összes ismétlődő együtt- vagy egymás ellen játszmát figyelembe veszi. Az asszortativitás mértéke e két módban eltér, és ez az eltérés számszerűen méri a kétféle gráfmod viselkedési különbségét.

2.4 Játékos-teljesítmény és MOBA-analitika

A League of Legends és az általánosabb MOBA-játékok esport-analitikája önálló kutatási területté vált. Mora-Cantalops és Sicilia átfogó irodalmi áttekintése megmutatja, hogy a MOBA-játékok egyedi kombinációt kínálnak verseny, együttműködés, játékosélmény és társas interakció között [?].

A játékos-teljesítmény modellezése Novak és munkatársai munkájában is megjelenik: profi League of Legends meccsekre kidolgozott teljesítménymodellek rámutatnak arra, hogy a domain-specifikus változók és a strukturált adatfeldolgozás fontosabbak, mint az általános statisztikai modellek [?]. Ez erősíti a jelen projekt szerepkör-érzékeny pontozási megközelítésének indokoltságát.

Sánchez-González és González-Bailón munkájában profi LoL-csapatok hálózati struktúráját kapcsolják össze a csapathatékonysággal [?].

2.5 Legrövidebb utak és heurisztikus keresés

A klasszikus legrövidebb út problémát Dijkstra alapozta meg [?], míg az A* keresés formális alapját Hart, Nilsson és Raphael adták meg [?]. A Rust rétegben az egzakt A* implementáció ALT-szerű alsó becsléseket használ, amelyeket a Goldberg–Harrelson-féle megközelítés ihletett [?]. A cél a játékos háló nagyságrendjéhez illeszkedő, helyes és stabil futtatókörnyezet, nem új algoritmus, hanem meglévő, tesztelt megközelítések megbízható alkalmazása.

2.6 Ágensorientált modellezés

Az ágensorientált modellezés (ABM) olyan szimulációs paradigma, amelyben az egyéni szereplők lokális szabályok alapján cselekszenek, és a makroszintű minták e döntések összegzésékként jelennek meg. Bonabeau és munkatársai ezt a keretrendszert természetes és mesterséges

6

rendszerekre egyaránt alkalmazták [?].

A dolgozat utolsó részét alkotó Tribal NeuroSim-kísérlet ebbe a paradigmába illeszkedik. Az ABM-irodalom hangsúlyozza: a szimulációból kapott eredményt a modell viselkedéseként kell értelmezni, nem a valóság közvetlen leképezéseként.

7

3. fejezet

Az adatgyűjtés és a Riot API

Az adatgyűjtés alapja a Riot Developer Portal és a match-v5 API-készlet [?, ?]. Ezek lehetővé teszik a játékosfiók azonosítását, a meccslisták lekérdezését, a részletes meccs-JSON-ok letöltését, valamint a résztvevői statisztikák feldolgozását.

3.1 A használt Riot API-vevők logikája

A jelenlegi projekt a Riot API több rétegét használja, de a legfontosabb tervezési döntés az volt, hogy a rendszer központi azonosítójává a puuid váljon. A pipeline logikailag az alábbi lépésekre bontható:

1. egy kiinduló játékos vagy seed-lista alapján a rendszer fiókinformációt kér le;
2. a fiókból meccsazonosító-listát szerez;
3. a meccsazonosítókhoz letölti a részletes match JSON-okat;
4. a résztvevői sorokból puuid-alapú játékosprofil és kapcsolati adatot képez;
5. az eredményt a datasethez kötve perzisztálja.

A collector az `europa.api.riotgames.com` regionális útvonalon hívja a `match-v5` végpontcsaládot.

3.2 Collector állapotgép és premade-probe

A `backend/match_collector.py` paraméterezhető állapotgép: konfigurációja dataset-azonosítót, útvonalakat, gyűjtési módot, queue-típust, meccsszám-korlátot, iterációs limitet és premade-küszöböt tartalmaz.

A 3.1 ábra a collector paraméterezhető állapotgépének teljes folyamatát mutatja be, az indítástól a perzisztenciáig.

8

Match Collector State Flow -- Rate Limits, Checkpoints, Seed Expansion, and Queue Filtering

Configure

Select collector config

Flex, SoloQ, or custom

JSON

Resolve dataset, queue,

limits, and seeds

RestoreRun

Restore frontier and

processed players

Continue from saved

checkpoint

StartFresh

Create initial frontier

from seed players

Write first checkpoint

Collecting

Base collection state

Take next player from

frontier

Fetch match ids and

match details

Respect Riot API rate

limits

RateLimited

Wait Retry-After or

configured delay

Keep current progress

RecoverableError

Log error

Skip or quarantine

invalid item

Write checkpoint if

needed

QueueFilter

Accept only configured

queue

Flex 440 or SoloQ 420

Reject duplicate or

incomplete matches

SaveMatch

Save raw match JSON

Update player stub

Write periodic

checkpoint

ExpandFrontier

Add newly discovered

players

Avoid processed,

queued, or invalid
PUUIDs
Write checkpoint
Finish
Write final checkpoint
Write collection
metadata
Dataset ready for
normalization and graph
building
yes no
HTTP 429
temporary
error or
invalid
payload
match
data
received
frontier
empty,
limit
reached,
or stop
requested
yes
no
yes
no

3.1. ábra: A collector paramétereizhető állapotgépének folyamatábrája

3.3 Gyűjtési fókusz és queue-választás

A projekt két kutatási datasetet használ: a flexset Ranked Flex Queue meccsekből áll, a solo pedig Ranked Solo/Duo Queue meccsekből. A queue-típus megválasztása nem tetszőleges. A Ranked Flex megengedi a premade csapatok részvételét, ezért a visszatérő szövetségi kapcsolatok természetes módon is megjelenhetnek. A Ranked Solo/Duo szándékosan kizárja az 5-fős premade-et, így kontrollált összehasonlítási alapként szolgál.

9

3.4 Sebességkorlátok és megbízhatóság

A Riot API korlátai az architektúrát is befolyásolják. A gyűjtőrétegben explicit sebességkorlátozás, várakozási logika és részfeladatokra bontás jelentek meg. A collector saját másodpercenkénti várakozást alkalmaz, HTTP 429 esetén harminc másodpercet vár, majd újrapróbálkozik, és a már elmentett meccseket fájllezés alapján kihagyja.

A 3.1 táblázat a collector öt fő megbízhatósági mechanizmusát és azok szerepét foglalja össze.

3.1. táblázat: A collector fő megbízhatósági mechanizmusai

Mechanizmus Funkció

Másodpercenkénti

késleltetés

A rövid távú API-terhelés kontrollja

Kétperces számláló Illeszkedés a Riot kulcsasznaleti limitjeihez

429 újrapróbálás Robusztusabb működés átmeneti limitütközéskor

Létező meccsfájl

kihagyása

Duplikált mentés és újra-letöltés elkerülése

Eseményes stdout log Backend- és frontend-oldali állapotkövetés támogatása

A fenti mechanizmusok nemcsak technikai részletek: a teljes flexset (közel 4980 meccs) gyűjtése fejlesztési API-kulccsal napokig tartott, megszakításokkal. A fájlalapú kihagyás és az újraindíthatóság nélkül a korpusz nem lett volna összerakható folyamatosan futó szerver nélkül. A collector tervezési döntése tehát közvetlen válasz az API korlátaira: iterált, állapotörző batch-futtatás, nem egy menetben történő letöltés. Egy-egy éjszakai futtatás után a kollekcio pontosan ott folytatódott, ahol abbahagyta.

3.5 A flexset és a solo gyűjtési stratégiájának összehasonlítása

A flexset dataset kizárólag 440-es queue ID-jű (Ranked Flex) meccseket tartalmaz, EUNEMaster+ ranglista-pozíciójú seed-játékosoktól. A collector premade-problem mechanizmusa kiszűri azokat a jelölteket, akiknek nincs elegendő visszatérő csapattársi mintázatuk.

A solo dataset 420-as queue ID-jű (Ranked Solo/Duo) meccsekből áll, EUNEMaster+ Solo/Duo ranglistás seed-populációval. A solo szerepe kontrollalap: olyan dataset, ahol a visszatérő

premade-kapcsolatok szisztematikusan nem favorizáltak.

A 3.2 táblázat a két dataset gyűjtési stratégiájának legfontosabb jellemzőit hasonlítja össze.

10

3.2. táblázat: A két dataset gyűjtési stratégiájának összehasonlítása

Dimenzió flexset soloq

Queue ID 440 (Flex) 420 (Solo/Duo)

Premade-probe Igen, ismétlések alapján Nem

Seed-terjeszkedés Csapattársak alapján Listalekéréssel

Gyűjtési cél Szociális struktúra Teljesítmény-alapvonal

Corpus mérete 4980 meccs 2000 meccs

Szerep a projektben Fő kutatási adatkészlet Kontroll-adatkészlet

A Riot API adatainak értelmezésekor szem előtt kell tartani, hogy a hálózat nem a teljes League of Legends univerzum reprezentációja, hanem egy adott gyűjtési stratégia által létrehozott, vizsgálható részvilág. A részletes adatminőségi és torzításelemzés a kibővített verzió 4. fejezetében található [?].

11

4. fejezet

A rendszer felépítése és evolúciója

4.1 A korai változat

Az eredeti rendszer jóval közelebb állt egy adatvezérelt játékosnézőhöz, mint a jelenlegi kutatási platform. A korai pontozási logika (add_new_players.js) egyszerű képletekkel dolgozott. A korai build_graph.py Python-központú volt: meccsekből épített kapcsolati gráfot, és HTML- és PyVis-exportokat készített. Ezen az úton váltak végül a klaszterprofilok a Tribal NeuroSim szimulációs bemenetének alapjává.

4.2 A jelenlegi rendszer

A rendszer több egymástól jól elkülönített rétegre épül. Komponensei: Python (adatgyűjtés, gráfépítés, legacy-exportok), SQLite (központi perzisztencia), Node.js/Express (API-héj és orchestráció), Rust (útvonalkeresés, gráfanalitika, globális artifact-export), React/Vite (frontend és vizualizáció), C# ASP.NET Core (különálló adatbázis-böngésző) és a Tribal NeuroSim szimulációs motor.

A komponensdiagram a kibővített verzió 5. fejezetében található [?]; a lite verzióban az általános leírást a szöveg tartalmazza.

4.3 A technológiai stack indoklása

Minden technológiai döntéshez volt alternatíva; az alábbiakban az egyes rétegek kiválasztásának főbb szempontjai, valamint a mérlegelt versenytársak szerepelnek.

Python (adatgyűjtés, szkriptreteg). Gazdag ökoszisztéma (Riot API-kliens, NetworkX, pandas), rövid iterációs ciklus, könnyen módosítható gyűjtési logika. Alternatíva: Go vagy Ruby szkriptekkel gyorsabb indulás lett volna lehetséges, de a tudományos ökoszisztéma (NetworkX, scipy) Pythonhoz kötött. Hátránya: gyengébb teljesítmény nagy gráfműveleteknél, ezt a Rust-réteg helyettesíti.

12

SQLite (perzisztencia). Szervermentes, önálló fájl, könnyű verziókezelés: a Python és a Rust egyaránt natívan támogatja. Alternatíva: PostgreSQL erősebb párhuzamos írást és komplex lekérdezéseket adott volna, de a projekt egyszerre egy felhasználós, lokálisan futtatható, bemutatható rendszer. A SQLite kompromisszuma ennél a hatókörnél kedvezőbb [?]. Hátránya: egyidejű írás nincs, és a séma módosítása migrációt igényel.

Node.js / Express (API-héj). A frontend (React) és a backend ugyanazon ökoszisztémán belül élnek; az Express könnyen proxyzza a Rust-binárist és a Python-szkripteket. Alternatíva: FastAPI (Python) logikusabb lett volna a Python-réteg mellé, de Node jobb aszinkron I/O-t adott a Riot API relay-ekhez, és a React/Vite dev-flow integrációja egyszerűbb lett. Hátránya: CPU-intenzív számítások nem a Node-ban futnak.

Rust (gráfanalitika, útvonalkeresés). Nulla futásidejű overhead, biztonságos memóriakezelés, Rayon párhuzamosítás, egzakt determinizmus [?, ?]. Alternatíva: C++ ugyanolyan teljesítményt adna, de a Rust borrow checker megelőzi az érvénytelen iterátor-hibákat, amelyek gráf-traversal kódban gyakoriak. Go könnyebb concurrency-modellt adna, de gyengébb cache-teljesítménnyel. Hátránya: meredekebb tanulási görbe, hosszabb fordítási idő.

React / Vite (frontend). Komponensalapú UI, nagy ökoszisztéma (MUI, Three.js), gyors hot reload. Alternatíva: Vue vagy Svelte kisebb bundle-t adna, de aMUI és a Three.js React-integrációja érettebb. Hátránya: overkill lenne egy egyoldalas demóhoz; a projekt több vizualizációs oldalt igényel, ami igazolja a frameworköt.

Az 4.1 táblázat az elvetett architektúráis megközelítéseket és elutasításuk indoklását foglalja össze.

4.1. táblázat: Elvetett megközelítések és indoklásuk

Alternatíva Rövid előny Miért nem ez lett a végső irány?

Minden Pythonban Gyorsabb kezdeti iteráció
A futásidejű gráfalgoritmusoknál és a nagyobb exportoknál gyengébb teljesítmény- és memóriakontroll
Minden Rustban Erős típusosság és egységes motor
A gyűjtés és a gyors kísérletezés jelentősen nehezebbé és lassabbá vált volna
Frontend-központú számítás
Egyszerűbb demoarchitektúra
A böngészőre tolta volna a számításigényes gráfmunkát, gyengítve a reprodukálhatóságot
Szerveres SQL-adatbázis
Erősebb többfelhasználós modell

A projekt lokális, bemutatható és kutatásorientált természete mellett az SQLite kedve-zőbb kompromisszum maradt

4.4 Több dataset kezelése

Az implementáció több, egymástól független datasetet kezel. A backend/data/datasets.json regiszter alapján minden dataset külön rendelkezik nyers adatbázissal, finomított adatbázissal, meccs-JSON-könyvtárral és cache-könyvtárral.

13

A dataset-választás nem áll meg a Node-szinten. A backend a kiválasztott datasetből környezeti változókat képez a Rust futtatókörnyezet felé (DATASET_ID, GRAPH_DB_PATH, PATHFINDER_MATCH_DIR, PATHFINDER_BIRDSEYE_CACHE_DIR).

A 4.1 ábra egy dataset teljes életciklusát mutatja be a Riot API-adatforrásoktól az analitikai kimenetekig.

Dataset Lifecycle from Riot API to Analytical Output

Riot API

Collection Raw DB Refined DB Graph Cache Analytical Result

collect matches store raw data clean and normalize build player graph run analytics

4.1. ábra: Egy dataset teljes életciklusának diagramja a Riot API-tól az analitikai kimenetig

4.5 A jelenlegi adatállapot

Az adatállapotot 2026. április 28-án a repository közvetlen olvasásával rögzítettük.

A 4.2 táblázat az aktív adatállapotot rögzíti 2026. április 28-i állapot szerint.

4.2. táblázat: A két dataset aktuális adatállapota (2026. április 28.)

Mérőszám flexset soloq

Meccs-JSON fájlok 4 981 2 001

Finomított játékosok 15 421 6 768

Klaszterek 599 140

Klasztertagok 5 741 1 538

A soloq dataset klaszterei jellemzően duopárokat jelölnek: mivel a Solo/Duo Queue maximálisan kétfős premade csapatokat enged, az ismétlődő kapcsolatokból kiemelkedő klaszterek döntő többsége két-három játékosra szűkül.

4.6 Perzisztencia és relációs modell

A klaszterek egy korábbi fázisban csak JSON-exportban léteztek; az adatbázis-integráció eredményeként első osztályú adatbázis-objektumokká váltak. A séma három központi táblára épül [?]: players, clusters és cluster_members.

A 4.2 ábra a fő SQLite adatbázis egyedkapcsolati diagramját mutatja, beleértve a players, clusters és cluster_members táblákat.

14

4.2. ábra: A fő SQLite adatbázis egyedkapcsolati diagramja

A 4.3 ábra a relációs modellt mutatja be, kiemelve a három tábla közötti kapcsolatokat.

15

playersrefined.db Relational Model

players

puuid : TEXT «PK»

names : TEXT
feedscore : REAL
opscore : REAL
country : TEXT
match_count : INTEGER
clusters
cluster_id : TEXT «PK»
cluster_type : TEXT
algorithm : TEXT
size : INTEGER
best_op : TEXT
worst_feed : TEXT
summary_json : TEXT
center_x : REAL
center_y : REAL
build_version : TEXT
updated_at : TEXT
cluster_members
cluster_id : TEXT «PK, FK»
puuid : TEXT «PK, FK»
is_bridge : INTEGER
is_star : INTEGER
is_best_op : INTEGER
is_worst_feed : INTEGER
role_json : TEXT
contains
belongs_to

4.3. ábra: Relációs modell a klaszter- és játékosperzisztenciához (players, clusters, cluster_members)

A rendszerben két klasztercsalád működik párhuzamosan. A python_population típusú klaszterek a Python-gráfépítési pipeline kimenetei: NetworkX-alapú modularitásmaximalizálás, közösségetekdetektálási perspektívával. A rust_pathfinding típusú klaszterek a Rust motor futásidejű klaszterezéséből jönnek: ally-kapcsolatokon alapuló, erős összefüggési szegmensek, maximum 20 taggal. A klaszterprofilok aggregált teljesítménymutatói egyben a Tribal NeuroSim szimulációjának genomszekvencia-bemenetét is képezik.

16

5. fejezet

A két adatkészlet összehasonlítása: flexset és soloq

Az elemzés két eltérő queue-típusból gyűjtött adatkészletre épül: a Flex Queue és a Solo/Duo Queue eltérő matchmaking-mechanikája eleve különböző gráfstruktúrát eredményez. Ha ugyanazt a pipeline-t mindkét dataseten lefuttatjuk, a különbség nem modellparaméter, hanem queue-mechanika.

5.1 A két dataset szerkezeti különbségei

A Flex Queue (440-es queue ID) csapatszedést tesz lehetővé: játékosok 2–5 főig premade csapatot alkothatnak. Ez azt jelenti, hogy a szövetségi pozícióban ismételtlen megjelenő játékos-párok egy része szándékolt ismétlődés. Az ally/enemy arány (73.17% ally a flexset-en) részben ennek a mechanizmusnak a következménye.

A Solo/Duo Queue (420-as queue ID) legfeljebb két fős premade-et engedélyez. Az ally/enemy arány (65.24% ally a soloq-on) alacsonyabb, ami összhangban van azzal, hogy a véletlenszerűbb meccsek kevesebb domináns szövetségi kapcsolatot hoznak létre.

Az 5.1. táblázat a két dataset alapvető méretmutatóit tartalmazza.

17

5.1. táblázat: A két dataset alapvető méretei

Mutató flexset soloq
Meccs-JSON állományok 4981 2001
Nyers játékosok 15 147 2822
Finomított játékosok 15 421 6768
Klaszterek (DB) 599 140
Klasztertagok (DB) 5741 1538
Látható csúcsok 5741 1538
Látható élek 18 548 4111
Átlagos élsúly 3.405 2.364
Szövetségi/ellenfél arány 73.17% / 26.83% 65.24% / 34.76%
Vizualizációs klaszterek 1531 870

Átlagos klaszterméret 3.750 1.768

Legnagyobb klaszterméret 20 20

A flexset átlagos klasztermérete (3,750) több mint kétszerese a soloq-énak (1,768): előbbiben jellemzően 3–4 fős csoportok, utóbbiban 1–2 fős duók dominálnak. A queue-mechanika közvetlenül magyarázza ezt: a Flex Queue-ban legfeljebb 5 fős premade csapatok mehetnek együtt, míg a Solo/Duo Queue-ban maximum kettő.

5.2 Összefoglalás

Az 5.2. táblázat a két dataset összehasonlítását foglalja össze a legfontosabb dimenziók mentén.

5.2. táblázat: A két dataset összehasonlításának összefoglalója

Dimenzió flexset soloq

Queue-mechanika Flex, premade-fókusz Solo/Duo, egyénközpontú

Átlagos élsúly 3.405 (sűrűbb) 2.364 (ritkább)

Ally arány 73.17% 65.24%

Átlagos klaszterméret 3.750 1.768

Háromszög-

struktúrák/csomópont†

4.95 1.03

Mag-periféria szerkezet‡ Hídgazdag asszociatív mag

és sok periférikus ally-sziget

Kisebb, véletlenszerűbb lo-

kális csoportok

Social-path opscore assz. 0.2346 0.1750

Battle-path opscore assz. 0.1461 0.1625

Feedscore battle-path assz. -0.0108 -0.0366

Kutatási szerep Asszociatív gráf és teljesítménykapcsolat

Teljesítmény-alapvonal,

kontroll

† Háromszög-struktúrák (triádok): ha az A játékos ismeri B-t és C-t, és B is ismeri C-t, ez egy zárt háromszög

háromszög a hálózatban. Több ilyen zárt háromszög erősebb csoportkohézióra utal. ‡ Mag-periféria (core-periphery): a gráf első magja sűrűn összekötött, a perifériás csúcsok csak néhány kapcsolattal rendelkeznek.

A flexset az asszociatív játékosgráf és a visszatérő szövetségi kapcsolatok természetesebb te-repe, a soloq pedig a teljesítmény-alapvonal és a kontroll-dataset szerepét tölti be. A battle-path opscore értéke a soloq-on magasabb (0.1625), mint a flexset-en (0.1461). A social-path módban a flexset erősebb (0.2346 vs. 0.1750), ami a premade-szövetségi mechanizmust tükrözi.

19

6. fejezet

Az opscore teljesítménymetrika

A gráfanalitikai elemzés alapanyagát egy értelmes, szerepkörérzékeny teljesítménypontszám adja. A League of Legends meccsek nyers adatai (ölések, halálok, arany, sebzés) önmagukban nem hordoznak elegendő szemantikát: egy support 2. halála nem ugyanolyan értelmű, mint egy ADC 2. halála. Az adatgyűjtési és PUUID-normalizálási mechanizmusok a kibővített verzió 4. fejezetében kerülnek részletezésre; itt az opscore metrika belső modelljét és formuláit tárgyaljuk.

6.1 A korábbi modell korlátai

A modell fejlesztése fokozatos módszertani finomítások eredménye volt. A kiindulópontot egy nyolc-artifactos rendszer adta, amely szerepkör-szorzókat alkalmazott, majd a kapott nyers összegüket az adatbázis percentilis-eloszlásához viszonyítva normalizálta a [0, 10] intervallumra.

Két alapvető gyengeség hamar felszínre került. Az adatbázis-relatív normalizálás problémája: ha az adatbázis túlnyomórészt nagyon erős játékosokat tartalmaz, a medián játékos is a skála közepére eső értéket kap. A halálokon alapuló közelítés korlátai: a korábbi rendszer nem tudott különbséget tenni a tank, a support és a feeder szerepe között.

6.2 A helyi, szerepkörérzékeny opscore modell

A jelenlegi opscore modell négy elv szerint épül fel: lokális, adatbázis-független értékelés; öt artifact tíz alkategóriával; logaritmusos bővítés az elit tartományban; megbízhatósági súlyozás rövid meccsekre.

6.3 Az öt artifact és alkategóriák

A 6.1 ábra az öt artifact és tíz alkategória teljes szerkezetét mutatja be, feltüntetve a szerepkör-érzékeny súlyozást.

20

O
ps
co
re
a
rt
if
ac
t-
sz
er
ke
ze
te
:
Ö
t
fő
d
im
en
zi
ó
.
Tí
z
al
ka
te
gó
ri
a
.
S
ze
re
pk
ör
ér
zé
ke
ny
s
úl
yo
k
A
:
C
o
m
b
a
t
lm
p
a
ct
b
a
s
e
=
2
·(
w
_f

c·
S
_f
c
+
w
—
d
p
·S
—
d
p
)
o
ve
rf
lo
w
a
=
0
.2
5
F
ig
h
t
C
on
ve
rs
io
n
w
t_
kd
a
=
k
ill
s
+
a
ss
is
ts
·w
_a
ss
is
t
k
p
=
w
t_
kd
a
/ m
ax
(1
, t
ea
m
_k
il

P/
JG
/M
ID
w
=
0
.5
5
B
O
T

w
=
0
.4
5
U
TI
L

w
=
0
.7
0
D
a
m
a
g
e
P
re
ss
u
re
d
p
m
=
t
ot
al
D
m
gT
oC
ha
m
pi
on
s
/g
am
e_
m
in
sh
ar
e
=
t
ot

al D m g / t e a m _t o t a l D m g d m g _p r e s s u r e = \sqrt{(S ^+(d p m / E _d p m) \cdot S ^+(s h a r e / E _s h a r e)) } TO P / J G / M I D w = 0 .4 5 B O T

.5
5
U
TI
L

w
=
0
.3
0
A
z:
R
is
k
D
is
ci
p
li
n
e
b
a
s
e
=
2
·(
w
_d
c·
S
_d
c
+
w
—
sq
·S
—
sq
)
o
ve
rf
lo
w
α
=
0
.2
5
D
ea
th
C
o
st
de
at
h_
ra
te
=

d
ea
th
s
/g
am
e_
m
in
de
ad
_t
im
e_
sh
ar
e
=
t
ot
al
T
im
eD
ea
d
/d
ur
at
io
n_
s
ri
sk
_r
at
io
=
(
0.
55
·d
r/
E
+
0
.4
5·
dt
s/
E
)/
(
1
+
0.
75
·k
p)
de
at
h_
co
st
=
S
~(

r
is
k_
ra
ti
o
)
—
—
—
—
—
—
—
—
—
—
—
TO
P/
JG
/M
ID
w
=
0
.6
5
B
O
T

w
=
0
.7
5
U
TI
L

w
=
0
.6
5
S
u
rv
iv
al
Q
u
al
it
y
li
vi
ng
_r
at
io
=
l
on
ge

st
T
im
eL
iv
in
g
/ d
ur
at
io
n_
s
cl
ut
ch
=
t
oo
kL
ar
ge
D
m
gS
ur
vi
ve
d

+
s
ur
vi
ve
d3
lm
m
ob
il
iz
es

+
s
ur
vi
ve
dS
in
gl
eD
ig
it
H
p
su
rv
iv
al
_q
=
0

.7
 0·
 S⁺
 (l
 iv
 in
 g_
 ra
 ti
 o/
 E
) +
 0.
 30
 ·S
⁺
 (cl
 ut
 ch
 /E
)

 TO
 P/
 JG
 /M
 ID
 w
 =
 0
 .3
 5
 B
 O
 T

W
=
0
.2
5
U
T
I
L

W
=
0
.3
5
A
3
R
es
o
u

rc
e
T
e
m
p
o
b
a
s
e
=
2
·(
w
_e
c·
S
_e
c
+
w
-
tp
·S
-
tp
)
o
ve
rf
lo
w
α
=
0
.2
5
E
co
n
o
m
y
g
p
m
=
g
ol
dE
ar
ne
d
/g
am
e_
m
in
sp
en
d_
ef
f=
c
la
m

p0
1(
go
ld
S
pe
nt
/
go
ld
E
ar
ne
d)
cs
p
m
=
(
m
in
io
ns
+
n
eu
tr
al
M
in
io
ns
) /
g
am
e_
m
in
ec
o
=
0
.5
5.
S
+(
gp
m
/E
) +
0
.2
5.
S
+(
cs
pm
/E
) +
0.
20
.s
pe
nd
_e
ff
—

—
—
—
—
—
—
—
—
TO
P
0
.5
5
JG
0
.4
5
M
ID
0
.6
0
B
O
T
0
.7
0
U
TI
L
0
.3
5
T
e
m
p
o
[L
an
er
]
0
.4
5·
S+
(c
s1
0/
E
)

+
0
.3
5·
I(
ea
rl
yG
ol
dE
xp

A
dv
)

+
0
.2
0·
l(
m
id
G
ol
dE
xp
A
dv
)
[J
un
gl
e]
0
.4
0·
S⁺
(j
gC
s1
0)
+
0
.3
0·
S
+(
cr
ab
s)

+
0
.3
0·
S⁺
(i
ni
ti
al
B
uf
fs
)
—
—
—
—
—
—
—
—
—
—

TO
P
0
.4
5
JG
0
.5
5
M
ID
0
.4
0
B
O
T
0
.3
0
U
TI
L
0
.6
5
A
4:
M
a
p
&
O
b
je
ct
iv
e
C
on
tr
ol
b
a
s
e
=
2
·(
w
_v
'
S
_v
c
+
w
_o
'
S
_o
c)
o
ve
rf
lo
w

α
=
0
.3
5
(
h
ig
h
es
t)
d
o
m
in
a
n
ce
r
ew
ei
g
h
ti
n
g
:
w
_o
'=
m
in
(0
.9
0
, w
-
o
+
0
.2
5
·d
o
m
)
V
is
io
n
C
on
tr
ol
vi
si
on
_p
m
=
v
is
io
nS
co
re
/

ga
m
e_
m
in
w
ar
d_
pm
=
(
pl
ac
ed
+
ki
ll
ed
+
de
te
ct
or
+
co
nt
ro
l)
/
m
in
vi
s
=
0
.7
0·
S⁺
(v
is
io
n_
pm
/E
)

+
0
.2
0·
S⁺
(w
ar
d_
pm
/E
)

+
0
.1
0·
cl
am
p0

1(
ct
rl
W
ar
dR
iv
er
C
ov
er
ag
e)

—

—

—

—

—

—

—

—

—

—

TO

P/

B

O

T

w

' =

0

.3

5

J

G

w

' =

0

.3

0

M

ID

w

' =

0

.4

5

U

TI

L

w

' =

0

.7

0

O

b

je

ct

iv

e

C

on

ve

rs

io
n
ob
jD
pm
=
d
m
gT
oO
bj
ec
ti
ve
s
/g
am
e_
m
in
ep
ic
=
d
ra
go
ns
+
1
.5
·b
ar
on
s

+
h
er
al
ds
+
2
·s
to
le
n
st
ru
ct
=
t
ur
re
ts
+
1
.5
·i
nh
ib
it
or
s
ob
j=
0

.4
0·
S⁺
(o
bj
D
pm
/E
) +
0
.3
5·
S⁺
(e
pi
c/
E
)

+
0
.2
5·
S⁺
(s
tr
uc
t/
E
)

—
—
—
—
—
—
—
—
—
—
—

TO
P/
B
O
T
w
' =

0
.6
5
J
G
w
' =
0
.7
0
M
ID

w
' =
0
.5
5

U
TI
L
w
'=
0
.3
0
A
s:
T
e
a
m
E
n
a
b
le
m
en
t
b
a
s
e
=
2
·(
w
_s
c·
S
_s
c
+
w
—
p
s·
S
—
p
s)
o
ve
rf
lo
w
a
=
0
.2
0
S
et
u
p
C
on
tr
ol
cc
_v
ol
=
(

im
m
ob
il
iz
at
io
ns
+
0
.2
5·
ti
m
eC
C
in
g)
/
ga
m
e_
m
in
cc
_c
on
v
=
im
m
ob
il
iz
eA
nd
K
il
IW
it
hA
ll
y

+
k
no
ck
ln
to
Te
am
A
nd
K
il
l
se
tu
p
=
0
.5
5·
S⁺

p
o
rt
al
ly
_p
ro
t=
e
ff
ec
ti
ve
H
ea
lA
nd
Sh
ie
ld
in
g

(f
al
lb
ac
k:
s
hi
el
ds
+
h
ea
ls
)
pr
ot
=
0
.7
5·
S⁺
(a
ll
y_
pr
ot
/m
in
/
E
)

+
0
.2
5·
S⁺
(s
av
eA

Il
yF
ro
m
D
ea
th
/
E
)
—
—
—
—
—
—
—
—
—
—
TO
P/
JG
w
=
0
.1
0
M
ID
w
=
0
.1
5
B
O
T

w
=
0
.2
0
U
TI
L
w
=
0
.4
5
o
p
sc
o
re
=
m
in
(1
0
, A
1 +
A
2 +
A

$\mathbf{A}^3 + \mathbf{A}^4 + \mathbf{A}^5)$
 $\mathbf{S}^{\text{CO}}$
 $\mathbf{r}^{\text{in}}$
 $\mathbf{g}^{\text{cu}}$
 $\mathbf{r}^{\text{v}}$
 $\mathbf{e}^{\text{r}}$
 $\mathbf{r}^{\text{=}}$
 $\mathbf{x}^{\text{/}}$
 $\mathbf{E}^{\text{r}}$
 $\mathbf{ol}^{\text{e}}$
 $\mathbf{B}^{\text{(r)}}$
 $\mathbf{r}^{\text{h/}}$
 $\mathbf{(r^h)}$
 $\mathbf{+m^h)}$
 $\mathbf{,m^=}$
 $\mathbf{(1-t)}$
 $\mathbf{/t)^{(1/h)}}$
 $\mathbf{S^+(r)}$
 $\mathbf{=m^{\text{in(c, B(r) + \lambda \cdot l_n(m_{ax}(1, r))})}}$
 $\mathbf{S^-(r)}$

=
S
+(
1
/r
)
t
=
0
.7
0
, h
=
2
, λ
=
0
.5
0
,
c
=
2
.2
0
A
_i
=
m
in
(4
.5
, A
_i
_b
as
e
+
α
i
R
_D
_f
ac
to
r
m
ax
(0
, A
_i
_b
as
e-
2)
)
6.
1.
áb
ra
:A
zo
ps
co
re
ar
tif

ac
t-s
ze
rk
ez
et
e:
öt
fő
di
m
en
zió
,tí
za
lka
te
gó
ria
,s
ze
re
pk
ör
ér
zé
ke
ny
sú
ly
ok
21

A modell öt artifactból épül fel. A Combat Impact a harckonverziót és a sebzési nyomást méri, a Risk Discipline pedig a halálok árát és a túlélési minőséget. A Resource Tempo a gazdasági hatékonyságot és a korai ritmust követi. A maradék két dimenzió a csapatszintű hatást fedi le: a Map and Objective Control a térkép- és objektívkontrollt, a Team Enablement pedig a csapattársaknak teremtett értéket.

6.4 Matematikai formalizmus

6.4.1 A szerepköri arány és az alapgörbe

Minden alkategória értékelésének kiindulópontja a megfigyelt érték és a szerepköri elvárás hányadosa:

$r =$
 x
Erole
(6.1)

Az alapgörbe egy kétparaméteres, S-alakú válaszfüggvény:

$m =$
(
 $1 - t$
 t
) $^{1/h}$
(6.2)
 $B(r) =$
 r^h
 $r^h + m^h$
(6.3)

ahol $t = 0,70$ és $h = 2$. A görbe pontosan azt biztosítja, hogy $B(1) = t$.

Az S-görbe két paramétere egy konkrét intuíciót kódol: aki pontosan az elvárásnak megfelelően teljesít ($r = 1$), mindig $B(1) = 0,70$ -et kap. Ez azt jelenti, hogy az elvárásszintű játékos a skála 70%-án van, nem küszöb alatti, de nem is kiváló. A logaritmikus bővítő tag ($\lambda \cdot \ln r$) azokat emeli meg, akik messze felülteljesítik a szerepköri elvárást: laposabb görbe nem tudná megkülönböztetni a 120%-on teljesítőt a 300%-on teljesítőtől.

Pozitív mutatóknál:

$S+(r) = \min(c, B(r) + \lambda \cdot \ln(\max(1, r)))$ (6.4)

ahol $\lambda = 0,50$ és $c = 2,20$. Negatív mutatóknál: $S-(r) = S+$

(

1
r
)
.

6.4.2 Időtartam-megbízhatóság

$RD = \text{clamp}[0,1]$

(
 $D - 12$
8
)

(6.5)

Ahol D a meccs hossza percben. Az ingadozásra érzékeny percenkénti mutatóknál:

$$\hat{S} = RD \cdot S + (1 - RD) \cdot 0,70 \quad (6.6)$$

22

6.4.3 Artifact-aggregáció és bővítés

$$A_{\text{alap}} = 2 (w_1 \cdot S_1 + w_2 \cdot S_2) \quad (6.7)$$

$$\gamma = 0,5 + 0,5 \cdot RD \quad (6.8)$$

$A_i = \min$

(
4,5, $A_{\text{alap}} + \alpha_i \cdot \gamma \cdot \max$
(
0, $A_{\text{alap}} - 2$
))

(6.9)

Az 6.1 táblázat az öt artifact bővítési paramétereit (α_i) tartalmazza.

6.1. táblázat: Artifact-specifikus bővítési paraméterek (α_i)

Artifact α_i

Combat Impact 0,25

Risk Discipline 0,25

Resource Tempo 0,25

Map and Objective Control 0,35

Team Enablement 0,20

6.4.4 A végső opscore

$\text{opscore} = \min$

(
10,
 $\sum_{i=1}^5$
 A_i
)

(6.10)

6.4.5 Kivételes teljesítmények és szélső esetek

A modell két jellegzetes szélső eseten keresztül szemléltethető.

Elité carry meccs. A fejlesztés során egy kiemelkedő TOP Quinn meccset vizsgáltak (kiugró ölésszám és magas assist), ahol a korábbi modell nyomás alá kerülhetett a kemény artifact-plafonok miatt. A jelenlegi modellben a Combat Impact és a Map and Objective Control artifactok logaritmikus bővítője lehetővé teszi, hogy ezek az értékek a nominális 2 pont fölé emelkedjenek; az összesített opscore a 10-es plafonig futott. A halálók ára nem tűnik el: a Risk Discipline artifact mérsékelten bünteti őket, de az élité carry összteljesítménye elismert marad.

Split-pusher és struktúracarry. Egy Yorick főszerepű játékos, aki szándékosan kerül a csatateret és struktúrákon keresztül nyer, tipikusan alacsony Team Enablement értékkel és nem kiemelkedő KDA-val rendelkezik. A Map and Objective Control artifact dinamikus dominancia-újrásúlyozása pontosan erre az esetre van kalibrálva: ha az Objective Conversion drámaian meghaladja a Vision Control értékét, az artifact súlyt vált az objektívumszerzés felé, és az alacsony csapaterősítő számnem torzírtja indokolatlanul a végeredményt, amennyiben awin condition-beváltás ténylegesen megvalósult. A részletes szélső esetek és mintanagysági korlátok a kibővített verzióban találhatók [?].

23

6.5 Az opscore szerepe a rendszerben

Az opscore és feedscore pontszámok két helyen jelennek meg a rendszerben. Az asszortativitás- és centralitásszámítások csomóponti attribútumként használják ezeket: a kérdés az, hogy az összekapcsolt játékosok hasonló teljesítményprofilokat hordoznak-e. A Tribal NeuroSim szimulációban a klaszter-szintű aggregált pontszámok (A_{combat} , A_{resource} , $A_{\text{map_objective}}$, A_{risk} , A_{team}) közvetlenül az öt artifactból számolódnak és a törzsek iniciális genomszekvenciáját alkotják; a meccsadatokból kiszámolt teljesítményprofil vezérli a szimulációban megjele-

nő viselkedési tendenciát.

24

7. fejezet

A játékosháló felépítése, keresése és mérései

7.1 A gráf felépítése

Az elsődleges gráfállapotot a Rust-réteg határozza meg. A Python oldal megőrzi az adatgyűjtés, a gyors prototipizálás és a kompatibilitási gráfgenerálás szerepét. A flexset és soloq összehasonlítása, az asszortativitás és a gráfvizualizáció Rust-alapú gráfmodellre támaszkodik.

7.1.1 Kapcsolataggregáció

A Rust gráfmodell minden játékospárra külön őrzi a szövetségi és ellenfél-oldali bizonyítékot:

$wally(u, v) =$

$\sum$

$m \in M$

$1[\exists i \in \{1, 2\} : u \in T_{m,i} \wedge v \in T_{m,i}]$ (7.1)

$wenemy(u, v) =$

$\sum$

$m \in M$

$1[\exists i \neq j : u \in T_{m,i} \wedge v \in T_{m,j}]$ (7.2)

A domináns reláció ally, ha $allyWeight \geq enemyWeight$, különben enemy. A gráf eredményei elsősorban asszociatív játékosgráfként, nem szó szerinti barátsági vagy ellenségességi hálóként értelmezendők.

7.1.2 A Rust gráfállapot

A Rust-réteg a gráfot közvetlenül a meccs-JSON-állományokból és a játékos-adatbázisból építi fel. A GraphState a szomszédsági listák mellett tárolja a pair_relations szerkezetet, a játékos-profilok gyors elérését, a klasztertagságot, a landmark-távolságokat, a minimumköltségeket és az aktív datasethez tartozó snapshot-metaadatot.

25

7.1.3 Szűrés és a kutatási gráf

A kutatási gráf láthatósági küszöbe $totalMatches \geq 2$: egyetlen közös mérkőzés még nem elég ahhoz, hogy egy kapcsolat stabil élként jelenjen meg. A klaszterezés fő iránya a Rust-oldali, determinisztikus bounded-ally-groups logika. A vizuális klaszterek mérete szándékosan korlátozott, legfeljebb körülbelül húsz játékosra.

7.1.4 A gráf artifact-rétege

A Rust gráf-export pipeline előkészített, datasethez kötött cache-artifaktumokat ír ki: a manifest.json rögzíti a datasetet és a gráfépítő verziót; a bináris fájlok (node_positions.f32, edge_pairs.u32) a Three.js/WebGL réteg bemeneti állományai; a cluster_meta.json a klaszterazonosítókat tartalmazza. Ez az artifact-réteg tekintendő a kutatási vizualizáció fő igazságforrásának.

7.2 Útvonalkeresés és vizualizáció

7.2.1 Keresési algoritmusok

A jelenlegi Rust futtatókörnyezet négy algoritmust támogat: BFS, Dijkstra, kétirányú keresés és A*. Az algoritmusok két értelmezésben futhatnak: a social-path csak szövetségi kapcsolaton, a battle-path pedig az összes ismétlődő kapcsolaton.

Az élköltség-modell: súlyozatlan módban minden él költsége fix 1000. Súlyozott módban:

$c(u, v) =$

[

1000

$w(u, v)$

]

(7.3)

ahol $w(u, v)$ az adott útvonalmódnak megfelelő kapcsolaterősség.

Az 7.1 táblázat a Pathfinder Labban a négy keresési algoritmus jellemzőit hasonlítja össze.

7.1. táblázat: A Pathfinder Labban szereplő algoritmusok összevetése

Algoritmus Mikor

természetes?

Megjegyzés a projektben

BFS súlyozatlan

legrövidebb út

Jó baseline, könnyen magyarázható

Dijkstra súlyozott utak Referenciaalgoritmus a költség-alapú módokban

Kétirányú

keresés

nagyobb gráf,

kevésbé

irányított

keresés

Gyakorlati gyorsítás bizonyos esetekben

A* jól kialakított

alsó becsléssel

A projektben egzakt marad, nem csak vizuális heurisztika

26

7.2.2 Egzakt A* és ALT heurisztika

AzALT (A* + Landmarks + Triangle inequality) heurisztika [?] elvéből indul ki: haL egy landmark-csomópont, akkor

$$d(u, t) \geq |d(L, t) - d(L, u)| \quad (7.4)$$

minden L landmarkra. A landmark-távolságok a GraphState felépítésekor előre kiszámíthatóknak

mindenModeKey-kombinációra, és az összes keresési kérésre újrafelhasználhatók. Az A* a helyes legrövidebb utat adja; egységteszték igazolják, hogy az A* eredménye minden releváns esetben egyezik a Dijkstra-eredménnyel.

7.2.3 Pathfinder Lab

A Pathfinder Lab a négy keresési algoritmus interaktív tesztkörnyezete: az eredmények lépésenként visszajátszhatók, a két útvonalmód (social-path, battle-path) közvetlenül összehasonlítható.

A 7.1 ábra a Pathfinder Lab replay panelét mutatja, amelyen az útvonalösszegző, a léptetési vezérlők és a bejárt csomópontok statisztikái láthatók.

7.1. ábra: A Pathfinder Lab replay panele: az útvonalösszegző, a léptetési vezérlők és a meglátogatott csomópontok statisztikái

27

7.2. táblázat: BFS és súlyozott keresés összehasonlítása: ugyanazon játékospár, eltérő közelségi fogalmak

Algoritmus Metrika Eredmény Értelmezés

BFS Minimális

ugrásszám

3 Legrövidebb szociális lánc

Dijkstra Min. súlyozott

költség

8 Legerősebb szövetségi útvonal

A* Min. súlyozott

költség

8 Egyezik Dijkstrával

Kétirányú Min. ugrásszám 3 Egyezik BFS-sel

A BFS által talált 3-ugrásos útvonal strukturálisan létezik, de gyenge vagy egyszeri meccsből eredő kapcsolatokon halad át. A Dijkstra súlyozott módban elkerüli ezeket, és helyette egy 8-ugrásos útvonalat talál, amelyen minden él ismétlődő szövetségi viszony.

Az 7.3 táblázat a négy algoritmus teljesítményjellemzőit és fő erősségeit hasonlítja össze.

7.3. táblázat: A négy algoritmus összehasonlítása a legfontosabb dimenziók mentén

Algoritmus Optimális? Memória Fő erősség a projektben

BFS Igen

(súlyozatlan)

O(n) Egyszerű baseline, jól

magyarázható replay

Dijkstra Igen

(súlyozott)

O(n+m) Referencia-algoritmus,

lexikografikus

döntetlen-kezelés

Kétirányú Igen

(súlyozatlan)

O(n) Potenciális gyorsulás

klaszteres gráfon

A* Igen

(heurisztika

elfogadható)

O(n) Célrányos bejárás,

egzakt eredmény

7.2.4 A 3D globális vizualizáció

A bird's-eye sphere a teljes hálózat vizuális áttekintésére szolgál. A Rust-réteg előre exportálja az összes fontos artefaktumot, a frontend pedig csak renderel.

Az 7.4 táblázat a bird's-eye export artefaktumait és azok szerepét mutatja be.

7.4. táblázat: A bird's-eye export fő artifaktumai

Fájl Szerep

manifest.json Metaadatok, darabszámok, skálázási és generálási információk
 node_meta.json Csomópontszintű leíró metaadatok, frontend-hozzárendelések
 node_positions.f32 Lebegőpontos koordináták tömbje a gyors rendereléshez
 node_metrics.u32 Tömörített numerikus csomóponti mutatók
 edge_pairs.u32 Él-végpontpárok tömbje
 edge_props.u32 Élhez tartozó kódolt tulajdonságok

A 7.2 ábra a gráf artifact-csomag teljes adatfolyamát mutatja be a Rust-exportortól a Three.js renderelőig.

Graph Artifact Package — Data Flow

Rust Runtime

backend/data/cache/{dataset Id} /graph-v2/

JSON / text Binary buffers

Frontend

engine/graph_v2.rs

graph_v2_export()

manifest.json

counts · scale · timestamp

node_meta.json

puuid · name · cluster · relation

cluster_meta.json

id · label · bridge · orbit

summary.md

stats · interpretation

node_positions.f32

x, y, z per node

node_metrics.u32

ally_weight · opscore

edge_pairs.u32

(src_idx, dst_idx)

edge_props.u32

flag · weight (scaled)

GraphV2Scene.tsx

Three.js · WebGL

Float32Array / Uint32Array

→ BufferGeometry

GraphSpherePage.tsx

dataset selector · controls

Analytics context

interpretation · export

Express API · server.js

/api/pathfinder-rust/graph-v2-export

/api/pathfinder-rust/graph-v2/*

writes JSON + markdown writes binary buffers

filesystem cache

binary buffers

(ArrayBuffer fetch)

manifest.json

node_meta.json

summary.md

manifest.json

rendered canvas

7.2. ábra: A gráf artifact-csomag adatfolyama: a Rust-exportőr bináris és JSON-rétegeket ír, az Express API kiszolgálja, a frontend WebGL-renderelőbe fogyasztja

A 7.3 ábra az artifact-csomag bináris és JSON-fájljainak szerkezetét mutatja be.

29

Graph V2 artifact-csomag szerkezete

Graph V2 Artifact Package

JSON leírók

Bináris payload (.u32)

manifest.json

node_meta.json cluster_meta.json edge_pairs.u32 edge_props.u32 node_metrics.u32

sor i -> edge_pairs[i] és

edge_props[i] azonos él

sor j -> node_meta[j] és

node_metrics[j] azonos csúcs

7.3. ábra: A gráf-export pipeline bináris és JSON artifact-fájljainak szerkezete

A 7.4 ábra a teljes flexset hálózat 3D madártávlati nézetét mutatja, az előre generált Rust-artifaktumok alapján renderelve.

7.4. ábra: A teljes hálózat 3D madártávlati nézete az előre generált Rust-artifaktumokra építve

A 7.5 ábrán a flexset gráf sűrűbb klaszterfoltjai, valamint a térbeli elkülönítés látható a globális nézetben.

30

7.5. ábra: Sűrűbb klaszterfoltok és a térbeli elkülönítés a globális nézetben

7.3 Asszortativitás

7.3.1 A számítási definíció

Az implementáció numerikus asszortativitást számol Pearson-korreláció formájában:

$r =$

$$\frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

$$(7.5)$$

$$\sqrt{\sum_i (x_i - \bar{x})^2}$$

$$\sqrt{\sum_i (y_i - \bar{y})^2}$$

(7.5)

Az irányítatlan éleket az implementáció mindkét endpoint-orientációval figyelembe veszi. A gráfmód alapján az él-súlyozás:

$w(u, v) =$

$\text{ally_weight} \text{ ha social-path}$
 $\text{total_matches} \text{ ha battle-path}$ (7.6)

7.3.2 Asszortativitási eredmények: flexset

Az 7.5 táblázat a flexset asszortativitási eredményeit tartalmazza kétféle gráfmódban.

31

7.5. táblázat: Asszortativitás: flexset

Gráfmód Metrika Korreláció Mintaszám

social-path opscore 0.2346 61 938

social-path feedscore 0.1816 61 938

battle-path opscore 0.1461 179 492

battle-path feedscore -0.0108 179 492

A 7.6 ábra a flexset social-path asszortativitási elemzésének vizualizációját mutatja opscore és feedscore szerint.

7.6. ábra: flexset social-path asszortativitás opscore és feedscore szerint. 15 421 jogosult csomópont és 61 938 elemzett él; globális együtttható opscore esetén 0.235, feedscore esetén 0.182.

7.3.3 Asszortativitási eredmények: soloq

Az 7.6 táblázat a soloq asszortativitási eredményeit tartalmazza, összehasonlítva a két gráfmódot.

32

7.6. táblázat: Asszortativitás: soloq

Gráfmód Metrika Korreláció Mintaszám

social-path opscore 0.1750 37 976

social-path feedscore 0.1012 37 976

battle-path opscore 0.1625 84 393

battle-path feedscore -0.0366 84 393

A 7.7 ábra a soloq social-path asszortativitási eredményeit mutatja.

7.7. ábra: soloq social-path asszortativitás opscore és feedscore szerint. 6 768 jogosult csomópont és 37 976 elemzett él; globális együtttható opscore esetén 0.175, feedscore esetén 0.101.

7.3.4 Értelmezés

Mindkét datasetben pozitív asszortativitás jelenik meg az opscore mentén a social-path módban, ami azt jelzi, hogy a szövetségesi kapcsolatokra épülő gráfban az összekapcsolt játékosok teljesítményprofilja nem véletlenszerűen keveredik. A feedscore viselkedése jóval érzékenyebb a gráfmódra: social-path módban mindkét dataseten mérsékelt pozitív jel látható, de battle-path módban közel nullára esik.

33

Az opscore minden vizsgált módban pozitív marad, a feedscore nem; ez a kompozit mutató erőssége az egyszerűbb halálközpontú képlettel szemben.

7.4 Betweenness centralitás

7.4.1 A Brandes-algoritmus

A Brandes-algoritmus [?] a hagyományos $O(n^3)$ komplexitású betweenness-számítást $O(nm)$ -re csökkenti. Forráscsomópontonként: Dijkstra/BFS futtatás, majd visszaterjesztési lépésben:

$\delta[s][v] +=$

$\sigma[s][v]$

$\sigma[s][w]$

$\cdot (1 + \delta[s][w])$ (7.7)

Súlyozott módban az élköltség:

$c(u, v) =$

[

106

$\max(1, w(u, v))$

]

(7.8)

7.4.2 Párhuzamos végrehajtás

ARust rayon könyvtára automatikusmunkalapós párhuzamosságot tesz lehetővé. A centralitás-modul a forráscsomópontok halmazát $k = \min(n, 4 \cdot \text{szálak száma})$ egyenlő méretű csomóponthalmazra osztja. A forráscsomópontok rendezése és a chunk-határok rögzítése garantálja, hogy azonos input esetén mindig ugyanaz a kimenet.

7.4.3 Eredmény

A 7.8 ábra a Brandes-algoritmus futáseredményének képernyőjét mutatja, beleértve a párhuzamos és soros futásidők összehasonlítását.

34

7.8. ábra: Brandes betweenness centralitás eredményképernyő. 15 421 runtime csomópontból 1 716 projektált csomópont és 2 766 elemzett él. Párhuzamos futás: 6.38 ms; soros baseline: 60.85 ms; gyorsulás: 9.54-szeres; 16 Rayon szál; soros-párhuzamos delta: 0.000000.

A 7.9 ábra a centralitás szempontjából kiemelkedő, közvetítő szerepű játékosokat mutatja a gráfban, sárgával jelölve.

35

7.9. ábra: A betweenness centralitás alapján azonosított broker játékosok a gráfban, sárgával kiemelve.

36

8. fejezet

Az adatbázis-alprojekt: PostgreSQL-séma és

C# böngésző

8.1 Az alprojekt háttere

A docs/databaseproject könyvtárban egy formálisan megtervezett, PostgreSQL-re épülő relációs adatbázismodell található. Ez az alprojekt kettős értékkel bír: erősíti az adatbázis-tervezés akadémiai oldalát, és valódi, futtatható eszközt biztosít az elemzési eredmények böngészéséhez.

8.2 A formális relációs séma

A docs/databaseproject/schema.sql egy teljes PostgreSQL-sémát definiál. A séma tíz főtáblája:

8.1. táblázat: A databaseproject PostgreSQL sémájának táblái

Tábla Szerepkör a sémában

players Játékosazonosság, összesített pontszámok, szerepkör

matches Meccsszintű adatok, queue, patch, időtartam, győztes

teams Egy meccs két csapata, oldal, objektíva adatok

match_participants Résztvevői adatok: champion, lane, kills, deaths, assists

player_relationships Játékos-játékos ally/enemy kapcsolat súllyal

clusters Klaszter metaadatok, típus, algoritmus

cluster_members Tagsági rekordok, szerepkör-összegzés

analysis_runs Elemzési futások metaadatai

performance_snapshots Játékos-teljesítmény pillanatfelvételek futásonként

path_queries Útkeresési kérések és eredmények

A nézeteket, tárolt eljárásokat és triggereket tartalmazó részek a kibővített verzió 12. fejezetében található [?].

37

8.3 Az ASP.NET Core / C# adatbázis-böngésző

A böngésző a Premade Graph SQLite-adatbázisainak böngészésére készült. Indulásakor a ProjectImporter.Import() folyamat importálja a fő rendszer adatbázisából a players, clusters, cluster_members és replay táblákat. A böngésző a repository datasets.json regiszterét olvassa be, és URL-paraméterként átadható a kívánt dataset-azonosító (?dataset=flexset, ?dataset=soloq). A böngésző szerveroldali HTML-generálással működik; nincs JavaScript a fronten. A Microsoft.Data.Sqlite NuGet-csomag az adatbáziselérés alapja. Az alkalmazás lefordul, és futtatható

a repositoryból.

8.4 Az alprojekt helye a rendszerben

A databaseproject nem pusztán papír-terv: a schema.sql futtatható, a C# böngésző lefordul és futtatható. Az SQLite-alapú fő rendszer a gráfépítés, az analitika és a demo-pipeline kiszolgálásához optimalizált, a PostgreSQL-alapú modell pedig a relációs adatbázis-tervezés akadémiai igényeihez igazodik. A kettő ugyanabban a domainben különböző célokat szolgál.

38

II. rész

Tribal NeuroSim

39

9. fejezet

Tribal NeuroSim: a szimuláció alapjai és felépítése

9.1 Célkitűzés és kutatási kapcsolat

A Tribal NeuroSim a PremadeGraph gráfelemzési pipeline közvetlen folytatása. A PremadeGraph pipeline minden egyes játékosklasztort teljesítménymetrikák halmazával jellemez: harci hatékonyság, erőforrás-gazdálkodás, objektívkontroll, kockázattűrés és csapatkoordináció. Ezek az értékek aggregált klaszterszintűmértékszámokká állnak össze (A_combat, A_resource, A_map_objective, A_risk, A_team), amelyek a Tribal NeuroSim törzseinek kezdeti genomszekvenciáját alkotják. A szimulációban egyidejűleg mindkét adatkészlet klaszterei megjelennek. A kutatási kérdés: ha Flex Queue-ból és SoloQ-ból derivált klaszterprofilok azonos szimulációs szabályrendszer alatt versenyeznek, vajon amögöttes gráfstruktúramérhető különbséget idéz-e elő az emergáló túlélési stratégiákban?

A kísérlet kerete exploratív transzfervizsgálat [?]. Nem kísérel meg közvetlen előrejelzést a valódi meccskimenetekre, és nem állít fel pszichológiai vagy szociológiai kauzalitást.

9.2 Ágensorientált és többügynök-modell

A Tribal NeuroSim ágensorientáltmodell (ABM): minden törzs önálló döntéshozó egység, amely kizárólag saját lokális állapota és közvetlen térbeli környezete alapján cselekszik. A makroszintű minták (terjeszkedés, háborúk, birodalomkialakulás, kihálás) kizárólag az egyéni döntések összegzésekként jelennek meg [?].

A rendszer többügynök-rendszerként (MAS) is értelmezhető [?]: minden törzs önálló ágensként működik saját belső állapottal, saját lokális percepcióval és saját döntési logikával (neurális hálózat). Az ágensek között nincs közvetlen kommunikáció: az interakció kizárólag a közös térbeli szubsztráton keresztül zajlik.

A szimuláció lefolyása azonos szerkezetű fázisokat mutat: korai terjeszkedés (tick 0-300), kon-

szolidáció (tick 300-1000), majd birodalmi háborúk (tick 1000+).

9.3 A rendszer felépítése

A 9.1 ábra a Tribal NeuroSim három rétegének hatáskör-megosztását mutatja be: Rust motor (autoritativ állapot), Node (közvetítő), MonoGame (megjelenítő).

9.1. ábra: A Tribal NeuroSim hatáskör-megosztása: a Rust motor az autoritativ szimulációs állapot forrása, míg a Node és MonoGame rétegek közvetítő és megjelenítő szerepet kapnak.

A Rust futtatókörnyezet az autoritás egyetlen forrása: a tick-ciklust, a világállapotot, a törzsek neurális döntéshozatalát, a háború- és szövetségfeloldást, a fuzionálási logikát, a tombstone-bejegyzéseket és az eseménynaplózást mind a Rust bináris kezeli [?]. A C# MonoGame kliens kizárólag vizualizációt és operátori felületet biztosít. A Node réteg vékony közvetítő: összekötőhíd a MonoGame és a Rust szerver között.

Ez a felosztás a böngészőalapú v2 prototípus közvetlen tanulsága: ahol a szimulációs állapot az UI-réteghez volt csatolva, ghost war jelenség, memóriaszivárgás és nem determinisztikus viselkedés keletkezett.

A ghost war konkrétan azt jelentette, hogy a befejezett és a halott törzsekhez tartozó háborúk soha nem kaptak lezárt státuszt: mivel nem létezett strukturált háborúlezárási esemény, a rendszer ezeket továbbra is aktívként tartotta nyilván, és a számuk a memóriában halmozódott: a korai prototípus végállapotában egyetlen élő törzs állt szemben több mint száz aktív szellemháborúval. A jelenlegi architektúrában minden döntés a Rust binárisban születik, és a törzs felszámolása atomikusan zárja le az összes aktív háborút (WarCancelled), így szellemháborúk nem keletkeznek; a MonoGame kliens csak a legutóbb kapott frame-et rendereli, és a szimuláció bármely pillanatban reprodukálható ugyanabból a kezdőállapotból.

9.4 FrameV1 bináris protokoll

A Rust szerver és a MonoGame kliens bináris WebSocket-kereteken kommunikálnak. Minden keret egy 40 bájtos TNS3 envelope-pal kezdődik (little-endian):

Az 9.1 táblázat a FrameV1 bináris WebSocket-keret 40 bájtos fejlécének mezőit mutatja be.

41

9.1. táblázat: FrameV1 desktop envelope fejléc.

Offset Méret Mező

0 4 B Magic: "TNS3"

4 2 B Version: 1

6 2 B HeaderBytes: 40

8 2 B PayloadKind: 2 (FrameV1)

10 2 B Flags

12 4 B PayloadBytes

16 8 B Tick (u64)

24 4 B Generation (u32)

28 4 B RecordCount (élő törzsek száma)

32+ változó V1 payload

A bináris formátum döntése teljesítményi: a Rust szerver 40ms alatt lekezel 599 törzs állapotát, miközben a MonoGame kliens aszinkron dekódol és renderel. A JSON-sorosítás elkerülése egy 599-törzsös világ esetén tickenként 150–200 kB felesleges szöveges overheadt spórol meg. Az eseményvezérelt megfigyelhetőség részletei és a determinizmus-hiba javítása (clusters.sort_by(|a, b| a.id.cmp(&b.id))) a kibővített verzió 13. fejezetében található [?].

42

10. fejezet

Tribal NeuroSim: játékklogika és neurális döntéshozatal

10.1 Neurális döntéshozatal és klaszter-priorok

Minden szimulált törzs egy neurális hálózatot hordoz, amelynek felépítése a NEAT (NeuroEvolution of Augmenting Topologies) paradigmáját követi [?]. A rendszer nem alkalmaz visszaterjesztést: a törzs nem kap tanítójelet egy valódi League of Legends-meccsel kapcsolatban. Ehelyett a fitness a szimulációs folyamat kimenetéből következik (túlélési időből, megszerzett területtől, populációméretből és elért polity-szintből), és minden ezer tickenként kerül kiértékelésre [?].

Minden törzs inicializálásakor valódimeccselőzményekből levezetett klaszterszintű teljesítménymutatókat kap iniciális genomszekvenciaként: A_combat, A_resource, A_map_objective, A_risk, A_team. Az inicializálás során minden törzshöz egy Genome::new(11, 7) struktúra jön létre 11 bemeneti csomóponttal, 7 kimeneti csomóponttal és 84 kezdeti kapcsolattal.

10.2 A 11 bemeneti jel és a 7 kimeneti hajtóerő

A genomból minden tickben következtetés indul el: a neurális hálózat 11 bemeneti jelet olvas be, és 7 kimeneti hajtóerőt ad vissza. A bemenetek egy része a törzs pillanatnyi állapotát tükrözi (élelmiszer, populáció, veszély), a másik része pedig a klaszterből örökölt, generációs határpontokon lassan mutálódó teljesítményprofil. A kimenetek közvetlenül a törzs viselkedési döntéseit modulálják: háborúindítás, terjeszkedés, szövetségkötés vagy visszavonulás.

Az 10.1 táblázat a neurális hálózat 11 bemeneti jelét mutatja be, megkülönböztetve a dinamikus (tick-szintű) és az invariáns (generációs) jeleket.

Az 10.2 táblázat a 7 kimeneti hajtóerőt és az aktiválási küszöbértékeket tartalmazza.

43

10.1. táblázat: A neurális hálózat 11 bemeneti jele.

Index Változónév Leírás Jelleg

0 food_ratio Élelmiszerkészlet / populáció Dinamikus

1 pop_ratio Populáció / maximális kapacitás Dinamikus

2 territory Igényelt tile-ok száma / 100 Dinamikus

3 feed_risk Éhhalál-közelség (1,0 = kritikus) Dinamikus

4 a_combat Harci teljesítmény-artifact Invariáns*

5 a_resource Erőforrás/gazdaság-artifact Invariáns*

6 a_map_objective Objektívkontroll-artifact Invariáns*

7 a_team Csapatkoordinációs artifact Invariáns*

8 nearest_enemy Legközelebbi ellenfél hex-távolsága Dinamikus

9 nearest_ally Legközelebbi szövetséges hex-távolsága Dinamikus

10 a_risk Kockázattűrési artifact Invariáns*

* Csak generációs határpontokon (1000 tickenként) frissül, mutáció révén.

10.2. táblázat: A 7 kimeneti hajtóerő és hatásuk a törzs viselkedésére.

Változónév Viselkedési hatás Aktiválási küszöb

aggression Háborúindítási valószínűség modulálása > 0,45

resource_drive Területterjeszkedési prioritás > 0,25

goal_drive Szövetségkezdemenyezés, polity-fejlesztés > 0,55 /> 0,50

migration_drive Migráció indítása (Settling állapotból) > 0,55

raid_drive Opportunista háború gyengébb szomszéd ellen > 0,35

isolation Szövetségajánlat elutasítása > 0,75

expansion_speed Tile-igénylési sebesség (hűtési idő módosítás) Folytonos

10.3 Fitness, mutáció és leszármazási öröklés

A fitness öt komponensből áll: survival_ticks, territory, population, polity_tier és war_record. A mutáció minden 1000. ticknél következik be. Az alapmutációs ráta $\mu_0 = 0,05$; a fitness-rangsor alapján ez differenciált:

$\mu_i =$

$2\mu_0$ ha $r_i \leq 0,20$ (leggyengébb 20%)

μ_0 ha $0,20 < r_i < 0,80$

1

2

μ_0 ha $r_i \geq 0,80$ (felső 20%)

(10.1)

ahol $r_i \in [0, 1]$ az i -edik törzs relatív fitness-rangszáma a populációban.

Fuzionálás feltétele mindkét félnél: goal_drive > 0,55, egyik sem tartózkodik AtWar állapotban, és isolation < 0,75. A fuzionálásból keletkező örökös genomja fitness-súlyozott keverék: `Genome::inherit_from(parent_a, parent_b, fitness_a, fitness_b)`.

Az első kísérleti futtatás (599 törzs, Flexset, 1200tick) kimenetiaktivitás- és fitness-differenciáció adatai, valamint a liveness-fix részletei a kibővített verzió 14. fejezetében találhatók [?].

A 10.1 ábra a klaszter-szintű teljesítménymutatókból felépített kezdeti genomszekvenciát vizu-
44

alizálja.

45

10.1. ábra: A genomgráf: a PremadeGraph klaszter-szintű teljesítménymutatóiból felépített kezdeti genomszekvencia vizualizációja. Az öt tengely (A_combat, A_resource, A_map_objective, A_risk, A_team) a neurális hálózat invariáns bemeneti jeleit alkotja.

46

11. fejezet

Tribal NeuroSim: kísérleti futtatások és eredmények

11.1 Világ-mechanikák

A szimuláció terét egy hexagonális rács alkotja. A flexset adatkészletből inicializált futtatások ≈ 36 100 tile-os térképen játszódnak; a soloq futtatások kisebb világokat kapnak. Minden tile-hoz egy biom-típus van rendelve (DenseForest, Riverland, DrySteppe, Mountains, Marsh, Cold), amely meghatározza az alap élelmiszertermelési rátát.

A törzsek öt polity-szinten fejlődhetnek: Tribe, City, Duchy, Kingdom és Empire. Minden szintlépéshez populáció-, terület- és szövetségesi küszöbértékek teljesítése szükséges; magasabb szinten több maximális populáció és jobb harci hatékonyság érhető el, de nagyobb élelmiszer-fenntartási igény is keletkezik. Az Empire-szint elérése a győzelmi feltétel: mind a négy teljes futtatás, mind a 103-seed futtatássorozat minden futtatása Empire-szintű győzelemmel zárult. A harci kimenet Poisson-folyamaton alapul: a várható veszteséget a támadó harci artifaktja és a kontaktzóna (a két terület határán szomszédos tile-párok száma) határozzák meg ($\lambda = a_{\text{combat}} \cdot n_{\text{kontakt}} \cdot 0,02$), de a konkrét tick-szintű kimenet véletlenszerű. A liveness-fix (2026. május 14.) három numerikus hiba javítása volt (a törzs-statisztikák kétszeres normalizálása, a migráció-hajtóerő oszcillációja és a túl konzervatív terjeszkedési küszöb), amelyek korábban statikussá tették a szimulációt. Ettől elkülönül a determinizmus-hiba javítása: a klaszterek rendezetlen SQLite-betöltése különböző tick-sorrenden át különböző kimenethez vezetett, a clusters.sort_by(id) bevezetése után pedig az újrafuttatások bit-azonos eredményt adnak. A részletes fejlődési feltételek és az iteratív tervezési döntések a kibővített verzióban találhatók [?].

11.2 A négy futtatás összefoglalója

A négy teljes kísérleti futtatás összefoglalója:

Az 11.1 táblázat a négy kísérleti futtatás főbb paramétereit és eredményeit foglalja össze.

47

11.1. táblázat: A négy záró Tribal NeuroSim futtatás összehasonlítása

Futtatás Adatkészlet Seed Győztes Viselkedés Agresszió Migráció Végső tick

run67f17777 Flexset 7777 Tribe 355 Warband/Combat 0,18 0,88 2538

run67f142 Flexset 42 Tribe 596 Warband/Combat 0,13 0,87 2487

run67sq42 SoloQ 42 Tribe 89 Vanguard/Risk 0,11 0,16 2046

run67sq7777 SoloQ 7777 Tribe 70 Supply/Resource 0,68 0,87 2541

A 11.1 ábra a flexset seed=7777 futtatásának végállapotát mutatja.

11.1. ábra: Flexset, seed=7777, tick=2538: Tribe 355 (Warband/Combat) győzelme, 37 666 tile-os végső területtel.

A 11.2 ábra a soloq seed=42 futtatásának végállapotát mutatja, ahol Vanguard/Risk profilú törzs nyert.

11.2. ábra: SoloQ, seed=42, tick=2046: Tribe 89 (Vanguard/Risk) győzelme, 6 356 tile-os végső területtel.

A 11.3 ábra a soloq seed=7777 futtatásának végállapotátmutatja, ahol Supply/Resource profilú törzs nyert.

11.3. ábra: SoloQ, seed=7777, tick=2541: Tribe 70 (Supply/Resource) győzelme, 9 748 tile-os végső területtel.

49

11.3 103-seed futtatássorozat és győztes archetípus-eloszlás

Anégy kísérleti futtatást egy 103-seed soloq futtatássorozat egészíti ki. Minden futtatás Empire-szintű győzelemmel zárult (103/103). Az archetípus-eloszlás:

Az 11.2 táblázat a 103-seed soloq futtatássorozat győztes archetípus-eloszlását tartalmazza.

11.2. táblázat: Győztes archetípus-eloszlás a soloq 103-seed futtatássorozat alapján

Archetípus Futtatások Arány

Warband 36 35,0%

Supply 23 22,3%

Council 20 19,4%

Vanguard 14 13,6%

Pathfinders 10 9,7%

Összesen 103 100%

Az átlagos befejezési tick 2308,57; az átlagos győztes terület 8 338 tile. A Warband archetípus enyhe pluralitással vezet (35%), de nem rendelkezik erős, seed-független dominanciával.

11.4 Mit mutatnak az eredmények

A két tesztelt flexset futtatás (seed=42 és seed=7777) mindkét esetben Warband/Combat profilú entitás győzelmével zárult, alacsony agresszióval (0,13–0,18) és magas migrációval (0,87–0,88). A soloq futtatások változékonyabb győztesprofilot produkáltak: seed=42 Vanguard/Risk, seed=7777 Supply/Resource győztest hozott.

Ez a különbség a mögöttes gráfstruktúrából értelmezhető. A flexset adatkészlet szervezettebb, ismétlődő kapcsolatmintákat hordoz, míg a soloq adatkészlet inkább egyéni teljesítményprofilokat rögzít tartós közös sorbeli struktúra nélkül.

Az értelmezés korrelatív, nem okozati. A determinisztikus batch-ellenőrzés helyességét igazolta: flexset seed=42 esetén öt párhuzamos, fejnélküli futtatás mindegyikében Tribe 596 (Warband/Combat) győzött, bit-azonos final_summary rekordokkal.

Azonos szabálykörnyezetben a flexset-priorokmindkét seedmellettWarband/Combat győztest adtak (agresszió 0,13–0,18), míg a soloq-priorok seedenként eltérő archetípust adtak (Vanguard, illetve Supply); a kezdeti klaszter-prior tehát mérhetően megváltoztatja a kimenetet. Az eredmények nem bizonyítanak valódi játékos-pszichológiát és nem adnak előrejelzést a tényleges League of Legends-meccsek kimenetelére.

50

12. fejezet

Összegzés

12.1 Mit mutat be a dolgozat?

A dolgozat egy olyan rendszert mutat be, amely valós League of Legends-mérkőzésadatokból indul ki, majd ezekből játékosprofilokat, kapcsolatokat, gráfokat, klasztereket, elemzési eredményeket és végül egy szimulációs kísérletet épít. A rendszer a játékosok ölés- és halálszámain túl azt is rögzíti, hogy egy játékos kikkel kerül egymecscsbe, kikkel kerül szembe, milyen ismétlődő kapcsolatok rajzolódnak ki körülötte, és ezekből milyen nagyobb hálózati minták olvashatók ki.

Ha valamit le kellene szűkíteni a projekt lényegéből, az inkább az egész lánc megléte, mint bármely egyedi komponensé. Egy Dijkstra-implementáció vagy egy React-oldal önmagában egy szokásos projekt. A rendszer érdekessége az összekötöttség: a nyers meccsadatokról a gráfon, klasztereken és Rust-analitikán át eljut egy reprodukálható szimulációs kísérletig, és minden lépés ellenőrizhető.

12.2 A fő eredmények

Az adatgyűjtési réteg két eltérő queue-típust kezel párhuzamosan. A flexset és a soloq külön kezelése azért fontos, mert a két queue nem ugyanazt a jelenséget méri: a Flex Queue alkalmasabb az ismétlődő kapcsolati minták vizsgálatára, a SoloQ pedig kontrollként és egyéni teljesítményprofilként értelmezhető.

Az opscore és feedscoremetrikák szerepkör-érzékeny pontszámokat adnak. Nem tökéletes igazságmérők, de lehetővé teszik, hogy a teljesítménymutatók gráfanalitikai bemenetként is használhatók legyenek.

A flexset gráf szerkezete core-periphery jellegű: vannak sűrűbb, erősebben összekapcsolt magrészek és sok lazább, kisebb periférius csoport. Ez a queue-mechanikából következik.

Az asszortatívási elemzés az opscore esetében pozitív kapcsolatot mutat (social-path: 0,235):

kapcsolatoknál váránk.

A Rust-alapú algoritmikus réteg (útvonalkeresés, asszortativitás, betweenness centralitás) determinisztikus és újrafuttatható: ugyanabból a bemeneti adatból mindig ugyanaz az eredmény jön ki.

12.3 A Tribal NeuroSim szerepe

A Tribal NeuroSim átviteli kísérlet: azt vizsgálja, hogy a gráf- és a klaszterekből kinyert teljesítményprofilok átvihetők-e egy evolúciós, többügynökös szimuláció kezdeti paramétereibe.

A négy futtatás alapján a Flex Queue-ból származó profilok egységesebb viselkedést mutatnak: mindkét vizsgált seed mellett hasonló, Warband/Combat jellegű győztes archetípust jelent meg. A SoloQ futtatások változókényabbak voltak, eltérő seedekkel eltérő győztes profilok alakultak ki. A flexset-priorok tehát egységesebb kimenetet adtak, a soloq-priorok változókényabbat, azonos szimulációs szabályrendszer mellett.

12.4 Visszatekintés: ami jól működött

Visszatekintve a legjobb döntés az volt, hogy a rendszer nem erőltette mindent egyetlen technológiába. A Python gyors adatgyűjtés és kísérletezésre maradt, a Rust ott vette át a szerepet, ahol teljesítmény és determinizmus kellett, a Node/Express és a React pedig bemutathatóvá tette az egészet. Ha mindezt egyetlen nyelvre kellett volna szűkíteni, valamelyik réteg szenvedett volna miatta.

A projekt erőssége az összefüggő lánc: a nyers API-adatoktól a normalizált játékosprofilokon, a gráfépítésem és a klaszterezésen át egészen az analitikai eredményekig minden lépés ellenőrizhető és újrafuttatható. Az asszortativitás, a betweenness centralitás és a core-periphery értelmezés nemcsak vizualizációt tesz lehetővé, hanem valódi kérdéseket is felvet a hálózatról.

12.5 Korlátok és nyitott kérdések

Az eredmények értelmezésekor néhány korlátot érdemes szem előtt tartani.

Az adatkészlet nem véletlenszerű mintavétel. A seed-játékosok kiválasztása, a premade-probe mechanizmus és a queue-típus együttmeghatározzák, milyen játékospopuláció kerül be a gráfba. A kapott szerkezet az adott gyűjtési stratégia terméke, nem általánosan érvényes EUNE Master+ leírás.

Az opscore és feedscore hasznos és értelmezhető mutatók, de nem külső igazságmérők. A szerepkör-elvárások és a görbe paraméterei a modell részei; más konstrukciós döntéssel más eredményt kapnánk.

A gráfban az ellenség-kapcsolat sem egyenlő valódi konfliktussal. Az ismételt szembekerülés jöhet a matchmaking MMR-sávjából, az azonos napszaktól vagy abból, hogy a Master+ Flex
52

Queue aktív játékospoolja eleve szűk. Ezért maradt az előjeles gráf kísérleti opciónak.

A temporal consistency és a community cohesion vizsgálatok kiestek a jelenlegi scope-ból. A kiesett vizsgálatok nem érdektelenek; az adatok és a módszer azonban nem tette lehetővé a védhető állításokat.

12.6 Záró

A projekt végeredménye egyszerűen megfogalmazható: kompetitív játékadatokból lehetséges olyan rendszert felépíteni, amely szoftvermérnöki szempontból is megállja a helyét és kutatói kérdéseket is tud feltenni. A PremadeGraph adatgyűjtési, adatmodellezési, gráfanalitikai és szimulációs láncát ezt igazolja.

A lépések közötti átjárhatóság nem volt előre tervezett. A nyers meccsadatokból normalizált játékosprofil lett, abból kapcsolati gráf, a gráf- és a klaszterekből analitikai mutatók, a klaszterekből szimulációs priorok; minden egyes lépés az előző kimenetét vette bemenetnek.

Ahogy az ABBA megfogalmazta évtizedekkel ezelőtt: the winner takes it all. A Tribal NeuroSimben ez nem metafora, hanem a mechanika közvetlen következménye.

12.7 Ahol ez máshol is jön számításba

A módszertan nem kötődik a League of Legends-hez. A Riot API azért volt kényelmes választás, mert nyilvánosan elérhető, gazdag és strukturált. De ugyanez a pipeline ugyanúgy lefutna GitHub-kollaborációs adatokon, NBA-teljesítménystatisztikákon vagy PubMed-szerzői hálózatokon, bárhol, ahol ismétlődő interakció, mérhető teljesítmény és valamilyen csoportszerkezet van.

A gráfklaszterezés például ugyanolyan természetesen leír egy vállalati szervezeti hálózatot, mint egy játékospopulációt: a kérdés mindkét esetben az, hogy a formális struktúra mögött milyen tényleges munkacsoportok rajzolódnak ki. A betweenness centralitás az infrastruktúra-tervezésben vagy az epidemiológiában is ugyanazt méri: melyik csomópont kiesése szakítja szét leginkább a hálózatot.

A Tribal NeuroSim öt artifaktja sem LoL-specifikus fogalom. Combat, resource, map_objective, risk, team: ezek más doménekben is lefordíthatók, és az evolúciós szimuláció keretrendszerében bármilyen klaszterprofilból kiindulhat, ahol csoportszintű teljesítménymutatók kinyerhetők. A videójáték itt eszköz volt, nem korlát.

